

Descriptor (Tanımlayıcı) Nedir ?

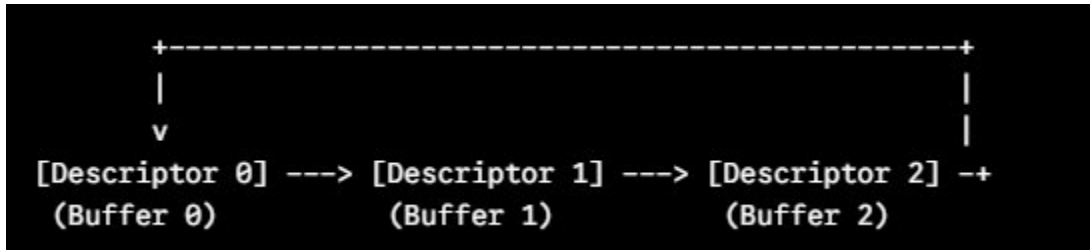
Bir **Descriptor**, STM32'nin RAM'inde yer tutan ve DMA'ya **reberlik eden bir C yapı (struct) bloğudur**. DMA, verinin kendisini doğrudan bilmez; sadece bu tanım bloklarını okuyarak ne yapacağını anlar.

Her bir Descriptor genelde **4 adet 32-bitlik kelimeden (16 Bayt)** oluşur ve temel olarak şu bilgileri tutar:

1. **Durum (Status) Bayrakları** : Bu paketin sahibi kim ? (CPU mu yoksa DMA mı?), Paket hatasız geldi mi ?, İlk veya son paket mi ?
2. **Buffer Boyutu (Control / Length)** : RAM'deki verinin boyutu kaç bayt ?
3. **Buffer Adresi (Buffer 1 Adress)** : Ethernet paketinin/verisinin RAM'de durduğu gerçek hafıza adresi pointer'ı.
4. **Sonraki Descriptor Adresi (Next Descriptor Address)** : Bir sonraki tanımlayıcının adresi (Zincir/Ring yapısını kurmak için).

Descriptor Zinciri / Halka Yapısı (Ring / Chain Structure)

Sürücü yazarken RAM'de tek bir Descriptor oluşturulmaz. Örneğin **4 adet TX (Gönderme)** ve **4 adet RX (Alma)** Descriptor'ı oluşturulur ve bunlar birbirine **dairesel bir zincir (Ring Buffer)** şeklinde bağlanır.



DMA bir Descriptor'ı işlediğinde (örneğin veriyi kabloya bastığında), bir sonraki Descriptor'ın adresine otomatik olarak atlar. Son Descriptor'a geldiğinde ise tekrar başa (0. Descriptor) döner.

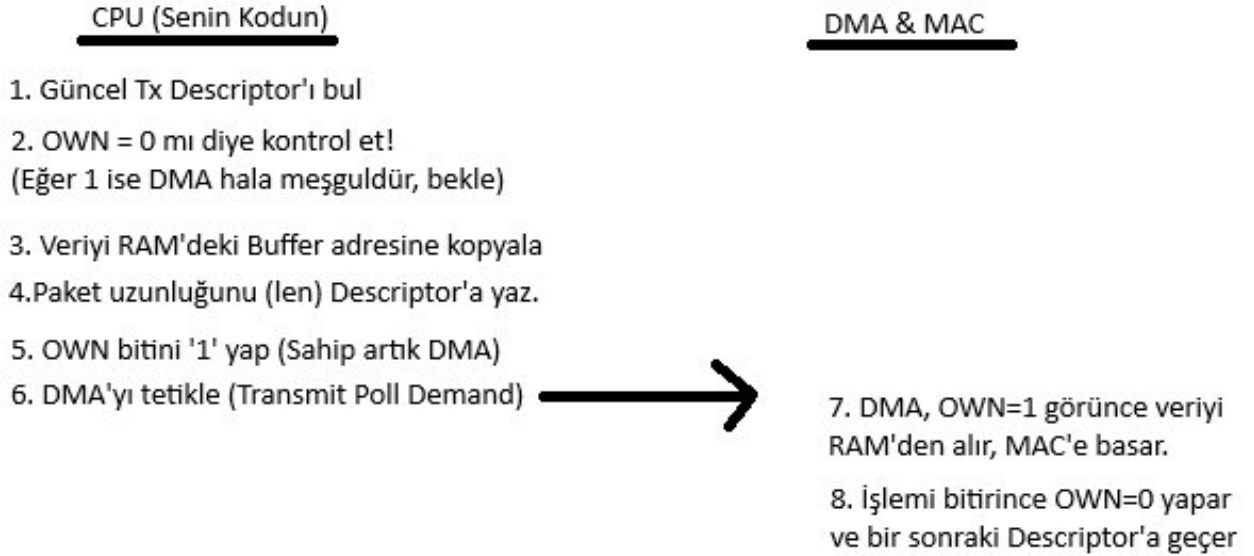
OWN (Sahiplik) Bitinin Yönetimi

Bir Descriptor içindeki en önemli bayrak OWN (**Bit 31**) bayrağıdır. Bu bit, o an o Descriptor ve ona bağlı RAM alanı üzerinde **kimin işlem yapma yetkisi olduğunu** belirler:

- **OWN = 1 -> Sahip DMA** : Bu alan donanımın kontrolündedir. CPU (senin kodun) bu alana kesinlikle dokunamaz, veri yazamaz veya okuyamaz.
- **OWN = 0 -> Sahip CPU** : Bu alan işlemcinin kontrolündedir. DMA bu alana dokunamaz. CPU yeni bir veri yazabilir veya gelen veriyi okuyup temizleyebilir.

Tx (Paket Gönderme) Mekanizması Nasıl Çalışır ?

ETH_SendPacket (uint8_t *data, uint16_t len) gibi bir fonksiyon yazdığında arka planda şu adımlar gerçekleşir.



RX (Paket Alma) Mekanizması Nasıl Çalışır?

Kablodan bir ping veya TCP paketi geldiğinde süreç tersine işler:

DMA & MAC

1. PHY ve MAC paketi yakalar.
2. Güncel RX Descriptor'a bakılır
3. OWN = 1 mi ? (Yani DMA yazabilir mi ?)
4. Evet ise paketi RAM'deki Buffer'a basar.
5. OWN bitini '0' yapar (Sahip artık CPU)
6. CPU'ya "Paket Geldi" kesmesi atar (Veya sen kodda anlık sorgularsın)



CPU(Senin Kodun)

7. CPU, güncel RX Descriptor'a bakar.
8. OWN = 0 olduğunu görünce veriyi RAM'deki Buffer'dan okur/LwIP'ye verir.
9. Buffer'ı temizler. OWN bitini tekrar '1' yapar (DMA'ya geri verir)

Aşama / Olay	OWN Biti Durumu	Sahip Kim?	CPU Ne Yapıyor?	DMA Ne Yapıyor?
TX - Hazırlık	OWN = 0	CPU	RAM'e gönderilecek veriyi yazıyor, uzunluğunu giriyor.	Beklemede / Diğer Descriptor'ları işliyor.
TX - Gönderim	OWN = 1	DMA	Diğer işlerine devam ediyor (Beklemek zorunda değil).	RAM'den veriyi okuyup PHY/MAC üzerinden kabloya basıyor.
RX - Bekleme	OWN = 1	DMA	Diğer işlerine devam ediyor / Paket gelmesini bekliyor.	Kablodan paket gelince RAM buffer'ına yazıyor.
RX - Okuma	OWN = 0	CPU	RAM'deki veriyi okuyup LwIP'ye (ağ yığınına) veriyor.	Bir sonraki Descriptor'a geçip yeni paket bekliyor.